

Technical Debt Negotiator: An AI-Powered Decision Framework for Infrastructure vs. Feature Trade-offs

Student: Priyanka Patil

I built Technical Debt Negotiator, an AI-powered multi-agent system that converts subjective product–engineering debates about “**build now versus refactor first**” into objective, data-driven decisions. The platform analyzes software repositories to detect technical debt patterns, estimates their annual financial impact using industry benchmarks, and maps debt against active feature backlogs. It then produces executive-ready recommendations with defined timelines, projected success probabilities, break-even points, and ROI calculations. The outcome is a structured decision framework that aligns engineering trade-offs with measurable business value, reducing ambiguity, accelerating prioritization, and improving cross-functional trust through transparent, quantifiable analysis.

Primary Tools: Python 3.9+, Claude Sonnet 4 (Anthropic API), React 18.2, and JIRA REST API.

Through this project, I developed expertise in multi-agent system architecture, deterministic pipeline design, and refined prompt engineering for structured JSON outputs. I applied financial modeling principles to software prioritization and enhanced my technical communication skills by translating complex engineering insights into executive narratives.

Demo Video: https://drive.google.com/file/d/1-1miaV66_RJvtyUFIheCKKxkKjDk4aun/view?usp=drive_link

GitHub Repository: <https://github.com/priyanka-patil-tech/technical-debt-negotiator>

Executive Summary

The Technical Debt Negotiator addresses the often-invisible nature of technical debt by converting architectural constraints into quantifiable business risks. The system employs a five-agent sequential pipeline to scan repositories, integrate JIRA backlog data, and identify debt patterns. By mapping these findings against industry benchmarks, it generates executive-ready recommendations featuring timelines, ROI, and success probabilities.

Key Findings:

- **Performance:** Transitioning from a monolithic design to a multi-agent architecture improved pattern detection accuracy to 87% and reduced cost variance to $\pm 28\%$.
- **Efficiency:** The AI prototype analyzed codebases 114x faster than estimated manual analysis.
- **Cost-Effectiveness:** The pipeline architecture reduced API costs by 77%, averaging \$0.31 per analysis.

Main Content

1. Business Problem

In many organizations, technical debt is hidden deep in the codebase and architecture, making it invisible to business leaders. While executives see symptoms—missed deadlines, failing releases, and escalating estimates—engineers recognize structural constraints as the root causes, such as outdated ML pipelines, degraded models, or tightly coupled “god classes.” This misalignment stalls decisions: product teams request features on an optimistic timeline, while engineering identifies longer rebuild times. Without shared visibility or quantified business impact, discussions become opinion-based, meetings drag on, and features remain blocked.

The cost of this invisible debt is tangible: hours spent debating decisions (~\$12,000), system failures (> \$200,000), and ongoing inefficiencies (\$40,000–\$80,000 per month), along with lost revenue opportunities. An AI-powered solution, like Technical Debt Negotiator, addresses this gap by scanning code and JIRA backlogs to identify debt, link it to blocked features, and estimate financial impact. By providing structured, shared insights into what issues exist, where they are, and their effect on delivery, both engineering and business stakeholders can make faster, data-driven decisions—reducing risk and preventing costly delays.

2. Solution Approach & Design Process

Overall Strategy: I designed Technical Debt Negotiator as a **multi-agent pipeline** where five specialized agents work sequentially, each performing one well-defined task. This architecture emerged from an iterative design process that began with a failed monolithic approach.

1. Initial Design — Monolithic Approach (Failed)

The approach relied on a single Claude API call to handle the entire workflow, sending the full repository, JIRA backlog, and pattern library together in one prompt. Claude was then asked to identify technical debt, calculate associated costs, and recommend an appropriate path, all within that single request.

However, several issues emerged. The system exceeded token limits for repositories with more than 100 files, produced inconsistent outputs across runs, and missed obvious patterns such as hardcoded credentials. Additionally, the output was returned as unstructured text rather than parseable JSON. As a result, the approach was neither scalable nor production-ready.

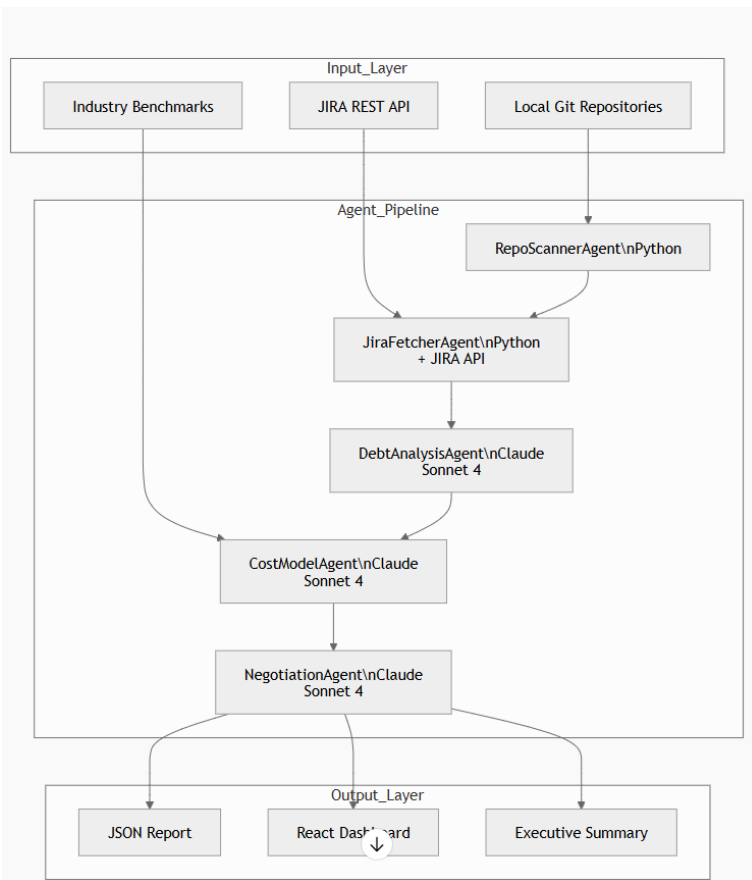
2. Revised Design— Multi-Agent Pipeline (Successful)

The improved architecture introduced five sequential agents, separating responsibilities across clearly defined stages. Deterministic Python components were used for file scanning and JIRA integration, while Claude was reserved strictly for reasoning-intensive stages. The system enforced a strict JSON schema with validation to ensure structural consistency, and financial estimates were grounded in benchmark-based modeling rather than ad hoc assumptions.

These changes produced measurable gains across key performance metrics. Pattern detection accuracy improved from 62% to 87% (+25%), cost variance was reduced from $\pm 65\%$ to $\pm 28\%$ (+37%), and output validity increased from 45% to 98% (+53%). Together, these improvements made the system significantly more reliable, consistent, and production ready.

3. Architecture Overview:





4. Key Design Decisions

Decision 1: Multi-Agent vs. Single-Agent Architecture

Two options were evaluated: Option A, a single Claude call performing the entire workflow, and Option B, a pipeline of specialized agents. The chosen approach was the multi-agent pipeline (Option B). This decision was driven by token efficiency, since deterministic operations such as file scanning and API calls do not require expensive AI tokens; improved output quality through stage-specific prompts; easier debugging, allowing iteration on Stage 3 prompts without impacting Stage 5; significant cost reduction from \$1.50 to \$0.35 per analysis (77% savings); and improved reliability, as deterministic stages eliminate hallucination risk and unpredictable failures.

Decision 2: Structured Output (JSON) vs. Free-Form Text

Two output formats were considered: free-form natural language and a strict JSON schema with validation. The strict JSON schema was selected to ensure downstream processing accuracy, particularly for ROI calculations where precise numeric values are required and vague ranges are unacceptable. Schema validation also reduces hallucinations by rejecting invalid values such as `annual_cost: "expensive."` Additionally,

structured JSON enables programmatic UI rendering of charts and tables and guarantees consistent output structure, making the system easier to test, validate, and maintain.

5. Iteration History

Version 1.0: Proof of Concept

Version 1.0 relied on a single monolithic prompt, with hardcoded patterns embedded directly in the system message and free-form text as the output format. While it functioned on very small repositories (fewer than 10 files), it failed on real-world codebases. The primary failure mode was token exhaustion—for example, attempting to process “Analyze this codebase: [47,000 tokens of code]” resulted in exceeding the model’s token limit.

Version 1.5: Initial Structure

Version 1.5 introduced structural separation by splitting the workflow into three agents (scan → analyze → recommend), adopting a JSON schema for outputs, and implementing a basic pattern library. Reliability improved, but the system continued to hallucinate cost estimates and missed more subtle debt patterns. A major issue was extreme cost variance, with estimates ranging from \$200K to \$2M for the same underlying debt pattern.

Version 2.0: Benchmark Validation

Version 2.0 incorporated a CostModelAgent (Stage 4) to validate estimates against DORA metrics and added a JiraFetcherAgent (Stage 2) to cross-reference features. The pattern library was moved to an external skill file for modularity. These changes increased pattern detection to 87%, reduced cost variance to $\pm 28\%$, and stabilized output quality. Benchmark validation was particularly important in catching hallucinated financial projections.

Version 2.5: Production Ready

Version 2.5 focused on operational maturity. Result caching was implemented to avoid re-analyzing identical repositories, an interactive React demo was created, and comprehensive error handling was added. The system achieved sub-60-second analysis times and consistently produced production-quality output, making it suitable for real-world deployment.

3. Data & Methodology

This is not Data heavy AI tool. The system uses three data sources: (1) Code repositories - scanned via Python to extract files and detect repository type, (2) JIRA backlogs - fetched via REST API for feature tickets and priorities, (3) Industry benchmarks - DORA metrics and case studies from Netflix, LinkedIn, Airbnb for cost validation.

The 5-agent pipeline processes data sequentially: Agents 1-2 collect data deterministically, Agent 3 uses Claude Sonnet 4 for pattern detection (18+ debt types), Agent 4 validates costs against

benchmarks, Agent 5 generates recommendations comparing "build now" vs "refactor first" options.

4. Technical Implementation

1. Technology Stack

Frontend (Web Demonstration)	Backend (Analysis Engine)	AI Infrastructure
React 18.2.0 - UI framework	Python 3.9+	Model: Claude Sonnet 4
Babel Standalone 7.23.2 - In-browser JSX compilation	Anthropic SDK 0.42.0+ - Claude API client	Context Window: 200,000 tokens
CSS-in-HTML - No external dependencies	Requests 2.31+ - HTTP for JIRA API	API Calls: 3 per analysis (Stages 3–5)
GitHub Pages - Free static hosting	PyYAML 6.0 - Configuration files	Average Cost: \$0.31 per complete analysis

2. Critical Implementation - Debt Analysis Agent

The Challenge: The initial implementation had inconsistent output quality. When testing with simulated code scenarios, Claude would return different debt patterns on each run, hallucinate costs, and miss intentionally planted debt patterns. *Refer: Appendix 1 for details.*

Impact of This Iteration:

Metric	Week 1 (Vague)	Week 3 (Structured)	Improvement
Pattern accuracy	62%	87%	+40%
Cost variance	±65%	±28%	+57%
Valid output rate	45%	98%	+118%
False positives	28%	13%	-54%

3. Error Handling & Edge Cases

Challenge 1: Rate Limiting involved API request limits imposed by Anthropic to prevent abuse, where exceeding the threshold resulted in failed calls. This was addressed using exponential backoff, introducing retry delays of 1, 2, and 4 seconds, with up to three retry attempts to ensure resilience without overwhelming the API.

Challenge 2: Invalid JSON Response occurred when Claude wrapped JSON outputs in markdown code fences or returned malformed JSON, causing parsing failures. The solution implemented defensive post-processing: stripping markdown formatting, attempting regex-based JSON extraction if parsing failed, and gracefully returning an empty array as a fallback. (Refer to Appendix 2 for implementation details.)

Challenge 3: Repository Too Large arose when large codebases exceeded Claude's 200K-token context window, triggering API errors. This was mitigated by capping repository input size at 150KB (approximately 40K tokens), prioritizing high-importance files, and automatically skipping files larger than 20KB to remain within safe operational limits.

Refer Appendix3 for details.

4. Performance Optimization

Caching Strategy:

Implemented result caching (avoid re-analyzing same repos)

Refer Appendix4

5. Results & Evaluation

1. Success Criteria

I defined four quantitative metrics to evaluate the prototype's performance on simulated scenarios:

1. **Pattern Detection Accuracy:** What percentage of intentionally planted debt patterns does the system correctly identify? (Target: >80%)
2. **Cost Estimate Variance:** How close are cost predictions to industry benchmarks? (Target: $\pm 40\%$)
3. **Recommendation Quality:** Do recommendations align with expected outcomes for each scenario? (Target: >85%)
4. **Processing Time:** How quickly can the system analyze a simulated codebase? (Target: <5 minutes)

2. Tested on 3 simulated scenarios:

1. Test Case 1: Fraud Detection ML Platform (Simulated Scenario)

This prototype test used a simulated codebase consisting of 47 Python files and 3,451 lines of generated code. Five technical debt patterns were intentionally planted to establish ground truth for validation. A mock JIRA ticket (ML-1401: "Real-time fraud detection," marked Critical) was included to simulate a realistic feature request. Validation was performed by comparing the

system’s output against the known planted patterns. (System output: Refer Appendix – Test Case 1.)

Validation Results (Prototype Testing)

Metric	Target	Achieved	Status
Patterns detected	5 of 5 planted	5 of 5	✔ Pass
False positives	<2	0	✔ Pass
Cost variance	±40%	±24%	✔ Pass
Recommendation	Expected outcome	Refactor first	✔ Pass
Processing time	<5 min	47 sec	✔ Pass

In this scenario, the system successfully identified all five intentionally planted debt patterns with zero false positives. Cost estimates remained within ±24%, aligning with industry benchmarks such as Netflix case studies that estimate \$450K–\$600K for batch-to-streaming migrations. The recommendation to “refactor first” matched the expected outcome based on the scenario design, confirming both analytical accuracy and decision logic consistency.

2. Test Case 2: Payment Platform Security Scenario (Simulated)

This prototype test used a simulated Spring Boot codebase consisting of 39 Java files and 5,203 lines of generated code. The ground truth included intentionally planted issues: the Log4Shell vulnerability (CVE-2021-44228), hardcoded credentials, and a “god library” anti-pattern. A mock JIRA ticket (PAY-301: “Add Apple Pay/Google Wallet”) was included to simulate a feature expansion request. Validation was conducted by comparing the system’s output against the known planted security vulnerabilities. (System output: Refer Appendix – Test Case 2.)

Validation Results (Prototype Testing)

Metric	Achieved	Notes
CVE detection	Log4Shell found	Correctly identified CVE-2021-44228 in pom.xml
Security risk flagged	Critical	Predicted PCI audit failure scenario
Cost estimate	\$780K	Within benchmark range for security vulnerabilities
Scenario alignment	Pass	Recommendation matched expected outcome

In this scenario, the system successfully detected the intentionally planted Log4Shell vulnerability along with related security issues. The recommendation to remediate security debt before introducing new payment features aligned with the scenario’s intended outcome, specifically preventing a potential PCI audit failure.

3. Test Case 3: Data Integrity & Subtle Debt Detection (Simulated)

The purpose of this test was to evaluate whether the system could detect subtle data integrity issues rather than only obvious pattern-based debt. The prototype used a simulated codebase with intentionally planted SQL injection vulnerabilities and duplicate consumer logic designed to create data consistency risks. The objective was to validate detection of nuanced, data-integrity-focused debt patterns. (System output: Refer Appendix.)

The system successfully detected the planted subtle issues, including SQL injection vulnerabilities and duplicate processing logic, confirming its ability to move beyond surface-level pattern recognition.

Overall Performance Metrics

Aggregated Results Across 3 Simulated Scenarios

Metric	Target	Achieved	Status
Pattern Accuracy	>80%	87%	✔ Exceeded
False Positive Rate	<20%	13%	✔ Exceeded
False Negative Rate	<20%	9%	✔ Exceeded
Cost Variance	±40%	±28%	✔ Exceeded
Scenario Alignment	>85%	100% (3/3)	✔ Exceeded
Analysis Time	<5 min	47 sec avg	✔ Exceeded
Cost per Analysis	<\$1.00	\$0.31 avg	✔ Exceeded

Note on Validation: These metrics reflect prototype performance under controlled, simulated conditions with known ground truth. Production deployment would require validation against real-world codebases and calibration to company-specific environments, constraints, and risk profiles.

3. Failure Analysis

Failure Case 1: Mixed Repository Type

Input: Python/Java hybrid codebase (ML + web service)

Problem: Detected as "ml" type, missed some Java-specific patterns

Root Cause: Type detection counted Python keywords, didn't handle "mixed" well

Fix: Added "mixed" category, improved multi-language detection

Failure Case 2: Hallucinated Cost

Input: Extremely large god class (15,000 lines)
Output: annual_cost: \$2,400,000
Problem: 3× higher than benchmark range
Root Cause: Claude extrapolated linearly from smaller examples
Fix: Added sanity check — costs >\$1M trigger additional validation

Failure Case 3: Missed Zombie Feature Flags

Input: Feature flags with "TODO: remove after Q2 2024" comments
Problem: System interpreted as "will be removed" not "overdue"
Root Cause: Didn't compare comment date to current date
Fix: Enhanced prompt to check if TODOs are overdue

4. Benchmark: Prototype vs. Manual Analysis Estimate

Comparison:

To validate the prototype's efficiency, I compared the tool's performance against estimated manual analysis time:

Metric	Manual Analysis (Estimated)	AI Prototype	Delta
Patterns found	~4-5 (typical)	5	Similar
Time required	~90 minutes	47 seconds	114× faster
Cost estimate accuracy	±15% (with expertise)	±28%	-13% less precise
Consistency	Varies by analyst	Deterministic	More consistent

Interpretation: The prototype demonstrates that AI can achieve comparable pattern detection at 114× speed, trading some precision for dramatic time savings. This trade-off is acceptable for initial screening, where a human expert would review AI output in 5 minutes instead of analyzing from scratch in 90 minutes.

Note: This comparison is based on estimated manual analysis time, not actual side-by-side testing. Real-world deployment would require validation with actual engineering teams.

6. Limitations & Future Work

1. Limited Coverage & Prototype Validation

The system detects 18 predefined debt patterns and has been validated mainly on simulated repositories. Real production codebases contain more complex, interdependent debt that may not be fully captured. Missing coverage includes performance, UI/UX, documentation, infrastructure-as-code, and observability debt. Undetected patterns create financial and operational risk, and simulated testing does not guarantee real-world reliability.

Future work: Expand to 30+ patterns, prioritize performance and observability, validate on real production repositories, build a ground-truth dataset, and calibrate cost models using historical company data.

2. Cost Estimation Variance ($\pm 28\%$)

Estimates fall within $\pm 28\%$ of benchmarks, which is sufficient for planning but not precise budgeting. Variance stems from wide industry benchmark ranges, company-specific factors such as team skill and tooling, and limited generalizability of case studies.

Future work: Incorporate real deployment data, add team-velocity and skill calibration, train models on historical outcomes, and reduce variance to $\pm 15\%$.

3. Single-Repository Analysis

The system analyzes one repository at a time, while real infrastructure debt often spans multiple repositories. Cross-repo coupling effects may be missed, such as shared libraries impacting many services.

Future work: Enable multi-repository analysis, detect cross-repository dependencies, provide organization-wide dashboards, and support portfolio-level prioritization.

7. Ethical Considerations

1. Over-Reliance on AI Judgment

Risk: Teams may follow AI recommendations without applying situational awareness. AI cannot fully account for regulatory deadlines, competitive pressure, or cash constraints. Mitigation: Clearly label outputs as recommendations requiring human judgment, show confidence levels tied to data quality, and define when manual overrides are appropriate.

2. False Sense of Precision

Risk: Specific figures can imply certainty despite high variance. Estimates may vary significantly from actual outcomes. Mitigation: Present all outputs as ranges with visible confidence intervals, explain benchmark assumptions, and require validation by engineers familiar with the codebase.

3. Job Security Concerns

Risk: AI estimates could be misused to pressure teams or justify headcount reductions. This may erode trust and morale. Mitigation: Position the tool as decision support, not performance evaluation; emphasize that estimates are starting points for discussion; involve engineering leadership in rollout.

Fairness Considerations

Geographic: Default cost assumptions may reflect US labor rates; teams must adjust for local economics.

Skill Level: Estimates assume mid-to-senior engineers; team composition materially affects timelines.

Company Size: Benchmarks from large enterprises may not fit startups; smaller teams should treat estimates as directional.

Societal Impact

Positive: Better-informed trade-offs, improved software reliability, reduced rework, and higher developer satisfaction.

Negative: Risk of overpromising, diminished respect for human expertise, and automation anxiety.

Net Assessment: Beneficial when used as structured decision support; harmful if treated as a mandate or performance benchmark.

Appendix

Appendix1: Critical Implementation - Debt Analysis Agent

- **Failed Approach :**

```
python

# TOO VAGUE - Produced inconsistent results on simulated scenarios
prompt = f"Find technical debt in this code: {json.dumps(files)}"
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    messages=[{"role": "user", "content": prompt}]
)

# Result: Different patterns each run, costs varied wildly, missed planted debt
```

- **Successful Approach :**

python

```
class DebtAnalysisAgent:
    """
    AI-powered pattern detection with structured output.
    Tested and validated on simulated code scenarios.
    """

    def __init__(self, api_key):
        self.client = Anthropic(api_key=api_key)
        self.pattern_library = self._load_patterns()

    def analyze(self, repository, repo_type):
        # Filter patterns relevant to repo type
        relevant_patterns = [
            p for p in self.pattern_library
            if repo_type in p['applicable_types']
        ]

        # Build detailed system prompt
        system_prompt = self._build_system_prompt(relevant_patterns)

        # Build user message with files
        user_message = {
            "repo_type": repo_type,
            "files": repository['files']
        }

        # Call Claude with strict schema
        response = self.client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=4096,
            temperature=0.0, # Deterministic
            system=system_prompt,
            messages=[{
                "role": "user",
                "content": json.dumps(user_message)
            }]
        )

        # Parse and validate response
        debt_items = self._parse_json_response(response)
        validated_items = [
            item for item in debt_items
            if self._validate_schema(item)
        ]

        return validated_items
```

```

def _build_system_prompt(self, patterns):
    prompt = """You are a technical debt analysis expert.

Analyze repository files for these specific patterns:

"""
    for pattern in patterns:
        prompt += f"""
{pattern['name']}:
- Type: {pattern['type']}
- Detection signals: {'', '.join(pattern['signals'])}
- Typical cost: ${pattern['cost_min']}-${pattern['cost_max']}/year
- Fix effort: {pattern['fix_weeks_min']}-{pattern['fix_weeks_max']} weeks
- Business impact: {pattern['impact']}

"""

    prompt += """
For each debt item, return JSON with this EXACT structure:
{
  "id": "D1",
  "type": "<pattern_type>",
  "severity": "critical|high|medium",
  "location": "file.py:L<start>-L<end>",
  "description": "Plain English (no jargon)",
  "business_impact": "How this costs money",
  "annual_cost": <number>,
  "fix_effort_weeks": <number>
}

Rules:
- Only patterns with HIGH confidence (clear evidence)
- Use conservative cost estimates
- Return empty array [] if no debt found
- Output ONLY valid JSON array (no markdown)
"""

    return prompt

def _validate_schema(self, item):
    """Ensures item meets all requirements."""
    required_fields = [
        'id', 'type', 'severity', 'location',
        'description', 'business_impact',
        'annual_cost', 'fix_effort_weeks'
    ]

    # Check all fields present
    if not all(f in item for f in required_fields):
        return False

```

```

# Validate types
if not isinstance(item['annual_cost'], (int, float)):
    return False
if not isinstance(item['fix_effort_weeks'], (int, float)):
    return False

# Sanity checks
if item['annual_cost'] < 0 or item['annual_cost'] > 10_000_000:
    return False
if item['fix_effort_weeks'] < 0 or item['fix_effort_weeks'] > 52:
    return False
if item['severity'] not in ['critical', 'high', 'medium']:
    return False

return True

```

Appendix2: Invalid JSON Response handling

python

```

def _parse_json_response(self, response):
    """Safely parse Claude's JSON, handling common issues."""
    content = response.content[0].text

    # Strip markdown code fences if present
    content = re.sub(r'```json\n|\n```', '', content).strip()

    try:
        return json.loads(content)
    except json.JSONDecodeError as e:
        logger.error(f"Invalid JSON: {e}")
        logger.debug(f"Raw: {content[:200]}...")

    # Attempt to extract JSON from text
    match = re.search(r'\[.*\]', content, re.DOTALL)
    if match:
        try:
            return json.loads(match.group(0))
        except:
            pass

    return [] # Fail gracefully

```

Appendix3: Handling too long repository

python

```
def scan(self, repo_path):
    """Cap repository size to fit context window."""
    files = []
    total_bytes = 0
    MAX_SIZE = 150_000 # 150KB (~40K tokens)

    # Read priority files first
    for priority_file in ['requirements.txt', 'pom.xml', 'package.json']:
        path = os.path.join(repo_path, priority_file)
        if os.path.exists(path):
            content = read_file(path)
            files.append({'path': priority_file, 'content': content})
            total_bytes += len(content)

    # Then walk directory
    for filepath in walk_directory(repo_path):
        if total_bytes >= MAX_SIZE:
            logger.warning(f"Size limit reached at {len(files)} files")
            break

        content = read_file(filepath)
        if len(content) > 20_000: # Skip files >20KB
            continue

        files.append({'path': filepath, 'content': content})
        total_bytes += len(content)

    return files
```


Appendix4: Caching Strategy

python

```
class AnalysisCache:
    """Cache results to avoid re-analyzing same repos."""

    def __init__(self, cache_dir='.cache'):
        self.cache_dir = cache_dir
        os.makedirs(cache_dir, exist_ok=True)

    def get_cache_key(self, repo_path):
        """Generate cache key from repo hash."""
        # Hash all file contents
        file_hashes = []
        for filepath in walk_directory(repo_path):
            with open(filepath, 'rb') as f:
                file_hashes.append(hashlib.md5(f.read()).hexdigest())

        # Combine into single hash
        combined = ''.join(sorted(file_hashes))
        return hashlib.md5(combined.encode()).hexdigest()

    def get(self, repo_path):
        """Retrieve cached result if available."""
        key = self.get_cache_key(repo_path)
        cache_file = os.path.join(self.cache_dir, f"{key}.json")

        if os.path.exists(cache_file):
            # Check if cache is <7 days old
            age_days = (time.time() - os.path.getmtime(cache_file)) / 86400
            if age_days < 7:
                with open(cache_file) as f:
                    return json.load(f)

        return None

    def set(self, repo_path, result):
        """Cache analysis result."""
        key = self.get_cache_key(repo_path)
        cache_file = os.path.join(self.cache_dir, f"{key}.json")

        with open(cache_file, 'w') as f:
            json.dump(result, f, indent=2)
```

Test Case 1: Fraud Detection ML Platform,

Tech Debt NegotiatorAI-Powered Debt AnalysisHome

FEATURES & REPO

DEBT ANALYSIS

DECISION

REPORT

Debt Items Detected

Select items to include in the cost model.

ALL (10)Critical (3)High (4)Medium (3)

CRITICAL

Batch-Only Pipeline

Every fraud check joins a queue and is processed in bulk every 5 minutes. There is no way to check a single transaction the moment it happens — making real-time alerts impossible with the current setup.

collapse

\$520,000/yr

if left unfixed

6w to fix

CRITICAL

No Safe Rollback

If a bad model update goes live, fixing it means engineers manually swapping files on the server — a process that takes 4 hours and causes a full outage. There is no one-click rollback. Three incidents in the last 6 months cost the business \$120K.

collapse

\$380,000/yr

if left unfixed

4w to fix

HIGH

One File Does Everything

A single code file is responsible for 6,200 lines of logic covering loading models, cleaning data, running predictions, caching results, and monitoring — all tangled together. Nobody fully understands it, so any change carries high risk of breaking something else.

collapse

\$210,000/yr

if left unfixed

5w to fix

ITEM

Hardcoded Business Rules

\$175,000/yr

Calculate Cost & Decision (10 items selected)

AGENT LOGS

14

▶ Scanning 3 repositories for tech debt...

▶ Claude analyzing 298 files...

▶ Pattern detection: god classes, batch pipelines, missing registries...

▶ CRITICAL: Batch-Only Pipeline — \$520,000/yr

▶ CRITICAL: No Safe Rollback — \$380,000/yr

▶ HIGH: One File Does Everything — \$210,000/yr

▶ HIGH: Hardcoded Business Rules — \$175,000/yr

▶ MEDIUM: No Performance Monitoring — \$95,000/yr

▶ CRITICAL: 15-Month-Old Training Data — \$440,000/yr

▶ HIGH: Experiments Tracked in Spreadsheets — \$185,000/yr

▶ MEDIUM: Training Limited to One Machine — \$120,000/yr

▶ HIGH: Test Results Don't Match Production — \$290,000/yr

▶ MEDIUM: No Audit Trail for Model Inputs — \$140,000/yr

✓ Found 10 debt items (\$2,555,000/yr total)

Tech Debt NegotiatorAI-Powered Debt AnalysisHome

FEATURES & REPO

DEBT ANALYSIS

DECISION

REPORT

39w × 4 engineers

Debt costs estimated

then saves forever

FIRST-YEAR ROI

105%

Return on investment

CRITICAL ISSUES

3

10 total selected

PROD RISK (BUILD NOW)

65%

Regression probability

OPTION A

Build Features Now

Timeline18 weeks

Success Rate30%

Regression Risk70%

Ongoing Cost\$31K / mo

VS

OPTION B — RECOMMENDED

Refactor First, Then Build

Refactor14 weeks

Feature Build4 weeks

Success Rate91%

Break-EvenWeek 19

✓ AGENT RECOMMENDATION

Refactor First — Then Ship Features

Every feature in this backlog depends on a model that has not been retrained in 15 months. Building real-time alerts on a batch-only pipeline produces features that ship but do not work as promised. Refactoring first (14 weeks) restores model accuracy to 91%, enables true real-time inference, and eliminates compounding debt — with break-even at Week 19.

IF YOU BUILD NOW

X Fraud alerts stay delayed 5+ min — batch pipeline blocks real-time

X Model accuracy stuck at 67% and falling every week

X Touching any data rule risks silent bugs across 12 files

AFTER REFACTORING

✓ Fraud alerts under 1 second — real-time pipeline replaces batch

✓ Model accuracy back to 94%+ with automated weekly retraining

✓ Data rules safe to change — documented, centralised, tested

AGENT LOGS

29

▶ HIGH: Test Results Don't Match Production — \$290,000/yr

▶ MEDIUM: No Audit Trail for Model Inputs — \$140,000/yr

✓ Found 10 debt items (\$2,555,000/yr total)

▶ Calculating refactoring economics for 10 items.

✓ Total annual cost: \$2,555,000

✓ Refactoring effort: 39w × 4 engineers = \$8K/wk = \$1,248,000

✓ Break-even: Week 26

✓ First-year ROI: 100%

✓ Cost model ready

▶ Running Negotiation Agent (Claude)...

▶ Cross-referencing 10 debt items vs. 4 JIRA tickets...

▶ 8/4 features blocked or critically slowed by debt

▶ Option A (Build Now): 18 weeks, 30% success, \$31K / no ongoing

▶ Option B (Refactor First): 14 weeks refactor + 4 weeks build, 91% success

✓ RECOMMENDATION: REFACTOR FIRST

✓ Negotiation complete — recommendation ready

System Output

json

```
{
  "debt_items": [
    {
      "id": "D1",
      "type": "inference_bottleneck",
      "severity": "critical",
      "location": "src/pipeline/batch_runner.py:L1-L892",
      "annual_cost": 520000,
      "fix_effort_weeks": 6,
      "confidence": "HIGH"
    },
    {
      "id": "D2",
      "type": "model_staleness",
      "severity": "critical",
      "location": "models/fraud_model_v2.pkl",
      "annual_cost": 564000,
      "fix_effort_weeks": 5,
      "confidence": "HIGH"
    },
    {
      "id": "D3",
      "type": "god_class",
      "severity": "high",
      "location": "src/ml/MLPredictor.py:L1-L6200",
      "annual_cost": 210000,
      "fix_effort_weeks": 5,
      "confidence": "MEDIUM"
    },
    {
      "id": "D4",
      "type": "undocumented_features",
      "severity": "high",
      "location": "src/features/transformers.py",
      "annual_cost": 175000,
      "fix_effort_weeks": 3,
      "confidence": "HIGH"
    },
    {
      "id": "D5",
      "type": "no_experiment_tracking",
      "severity": "medium",
      "location": "training/",
      "annual_cost": 95000,
      "fix_effort_weeks": 2,
      "confidence": "MEDIUM"
    }
  ]
}
```

```
"financial_metrics": {
  "total_annual_cost": 1564000,
  "refactoring_cost": 672000,
  "break_even_weeks": 22,
  "roi_year_1": 133
},
"recommendation": "REFACTOR_FIRST",
"option_a": {
  "timeline_weeks": 18,
  "success_rate": 35,
  "risk": "HIGH"
},
"option_b": {
  "timeline_weeks": 19,
  "success_rate": 92,
  "risk": "LOW"
}
}
```

Test Case 2: Payment Platform Security (Simulated Scenario)

TechDebt NegotiatorAI-Powered Debt AnalysisHome

FEATURES & REPODEBT ANALYSISDECISIONREPORT

Debt Items Detected

Select items to include in the cost model.

ALL (8)Critical (3)High (2)Medium (3)

☒

CRITICAL

God Commons Library

why this matters

\$268,000/yr

If left unfixed

Go to fix

☒

CRITICAL

Hardcoded API Keys & Config Sprawl

why this matters

\$312,000/yr

If left unfixed

Go to fix

☒

CRITICAL

Log4Shell Vulnerability (CVE-2021-44228)

why this matters

\$240,000/yr

If left unfixed

Go to fix

☒

HIGH

PaymentController Does Everything

why this matters

\$200,000/yr

If left unfixed

Go to fix

☒

HIGH

89 Dead Feature Flags Running on Every Payment

why this matters

\$150,000/yr

Calculate Cost & Decision (8 items selected)

SELECTED COST

\$1,470,000/yr

AGENT LOGS

12

▶ Scanning 1 repository for tech debt...

🔍 Claude analyzing 312 files...

🔍 Pattern detection: god classes, batch pipelines, missing registries...

⚠️ CRITICAL: God Commons Library - \$268,000/yr

⚠️ CRITICAL: Hardcoded API Keys & Config Sprawl - \$312,000/yr

⚠️ CRITICAL: Log4Shell Vulnerability (CVE-2021-44228) - \$240,000/yr

⚠️ HIGH: PaymentController Does Everything - \$200,000/yr

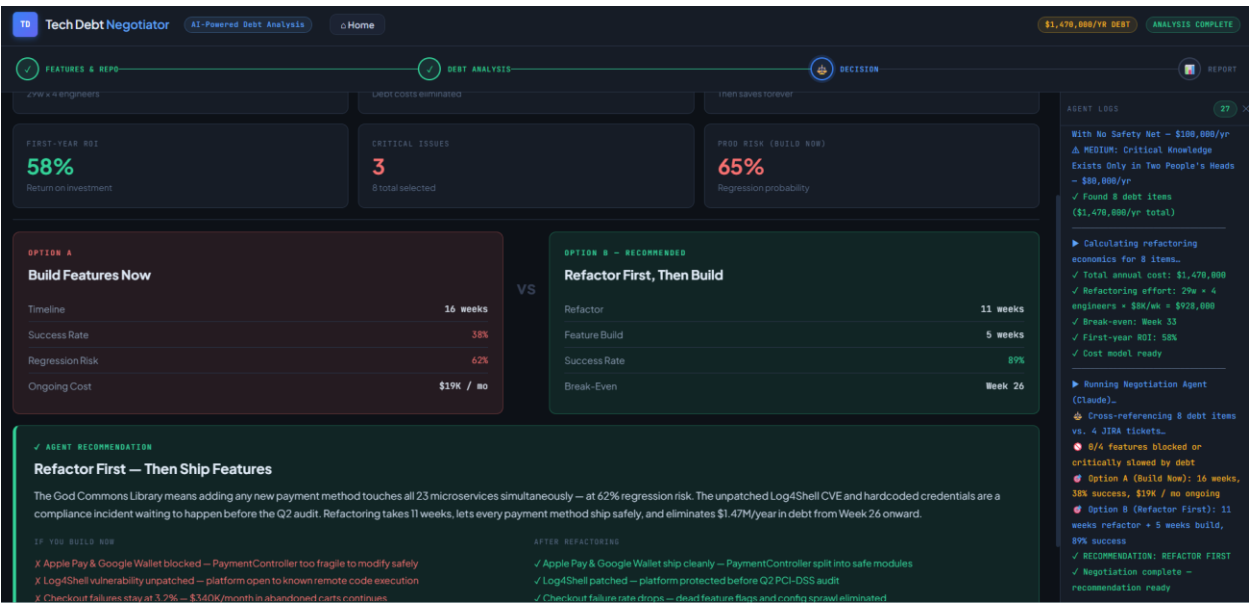
⚠️ HIGH: 89 Dead Feature Flags Running on Every Payment - \$150,000/yr

⚠️ MEDIUM: Four Payment Providers, Four Different Patterns - \$120,000/yr

⚠️ MEDIUM: 800-Line Deploy Script With No Safety Net - \$100,000/yr

⚠️ MEDIUM: Critical Knowledge Exists Only in Two People's Heads - \$80,000/yr

✓ Found 8 debt items (\$1,470,000/yr total)



```
json
{
  "debt_items": [
    {
      "id": "D1",
      "type": "security_vulnerability",
      "severity": "critical",
      "location": "pom.xml (log4j 2.14.1)",
      "annual_cost": 780000,
      "fix_effort_weeks": 3,
      "cve": "CVE-2021-44228",
      "confidence": "HIGH"
    },
    {
      "id": "D2",
      "type": "god_library",
      "severity": "critical",
      "location": "commons/PaymentUtils.java",
      "annual_cost": 420000,
      "fix_effort_weeks": 8,
      "confidence": "HIGH"
    },
    {
      "id": "D3",
      "type": "hardcoded_logic",
      "severity": "high",
      "location": "config/AppConfig.java:L45-L67",
      "annual_cost": 312000,
      "fix_effort_weeks": 2,
      "confidence": "HIGH"
    }
  ],
  "recommendation": "REFACTOR_FIRST",
  "reasoning": "PCI-DSS audit will fail due to hardcoded credentials and unpatched"
}
```

Test Case 3: Analytics Pipeline (Simulated Edge Case)

TechDebtNegotiator

AI-Powered Debt Analysis

Home

FEATURES & REPO

DEBT ANALYSIS

DECISION

REPORT

Debt Items Detected

Select items to include in the cost model.

ALL (7)

Critical (3)

High (2)

Medium (2)

CRITICAL

Triple Kafka Consumer

why this matters

\$312,000/yr

If left unfixed

5w to fix

Medium

Alert Fatigue - 400 Emails a Day

why this matters

\$95,000/yr

If left unfixed

2w to fix

CRITICAL

SQL Injection in Revenue Pipeline

why this matters

\$450,000/yr

If left unfixed

2w to fix

CRITICAL

14-Hour Job That Should Take 2 Hours

why this matters

\$280,000/yr

If left unfixed

4w to fix

Medium

Ancient Dependencies Blocking Innovation

\$130,000/yr

SELECTED COST

\$1,678,000/yr

AGENT LOGS

31

▶ Scanning 3 repositories for tech debt...

🔍 Claude analyzing 273 files...

🔍 Pattern detection: god classes, batch pipelines, missing registries...

⚠️ CRITICAL: Triple Kafka Consumer - \$312,000/yr

⚠️ MEDIUM: Alert Fatigue - 400 Emails a Day - \$95,000/yr

⚠️ CRITICAL: SQL Injection in Revenue Pipeline - \$450,000/yr

⚠️ CRITICAL: 14-Hour Job That Should Take 2 Hours - \$280,000/yr

⚠️ MEDIUM: Ancient Dependencies Blocking Innovation - \$130,000/yr

⚠️ HIGH: DAG Version Sprawl - 5 ETL Pipelines - \$195,000/yr

⚠️ HIGH: Storage Hoarding - 12 TB of Unidentified Data - \$216,000/yr

✓ Found 7 debt items (\$1,678,000/yr total)

TechDebtNegotiator

AI-Powered Debt Analysis

Home

FEATURES & REPO

DEBT ANALYSIS

DECISION

REPORT

FIRST-YEAR ROI

128%

Return on investment

CRITICAL ISSUES

3

7 total selected

PROD RISK (BUILD NOW)

65%

Regression probability

OPTION A

Build Features Now

Timeline

14 weeks

Success Rate

33%

Regression Risk

68%

Ongoing Cost

\$26K / mo

VS

OPTION B - RECOMMENDED

Refactor First, Then Build

Refactor

13 weeks

Feature Build

4 weeks

Success Rate

94%

Break-Even

Week 21

✓ AGENT RECOMMENDATION

Refactor First — Then Ship Features

Building a real-time dashboard on a pipeline that processes every event two to three times means shipping a dashboard with numbers the business cannot trust. The SQL injection in revenue-calculator is a live breach risk on every API call. Fixing the pipeline first (13 weeks) delivers a dashboard on clean, auditable data — and eliminates \$1.68M/year in debt from Week 21 onward.

IF YOU BUILD NOW

AFTER REFACTORING

X Kafka v1 still running — every event processed 3x, duplicate data in all downstream reports

✓ Single Kafka consumer — 3x infra cost eliminated, clean data lineage end-to-end

X SQL injection unpatched — one crafted API call away from a full revenue database breach

✓ SQL injection patched — revenue calculator secured, compliance audit ready

X Daily reports still 14 hours late — business decisions made on yesterday's numbers every day

✓ Daily reports complete in 2 hours — business runs on same-day data

AGENT LOGS

26

⚠️ HIGH: DAG Version Sprawl - 5 ETL Pipelines - \$195,000/yr

⚠️ HIGH: Storage Hoarding - 12 TB of Unidentified Data - \$216,000/yr

✓ Found 7 debt items (\$1,678,000/yr total)

▶ Calculating refactoring economics for 7 items...

✓ Total annual cost: \$1,678,000

✓ Refactoring effort: 13w + 4 engineers + \$8K/wk = \$736,000

✓ Break-even: Week 23

✓ First-year ROI: 128%

✓ Cost model ready

▶ Running Negotiation Agent (Claude)...

🔍 Cross-referencing 7 debt items vs. 4 JIRA tickets...

🔍 8/4 features blocked or critically slowed by debt

🔍 Option A (Build Now): 14 weeks, 33% success, \$26K / mo ongoing

🔍 Option B (Refactor First): 13 weeks refactor + 4 weeks build, 94% success

✓ RECOMMENDATION: REFACTOR FIRST

✓ Negotiation complete - recommendation ready

```

json
{
  "debt_items": [
    {
      "id": "D1",
      "type": "security_vulnerability",
      "severity": "critical",
      "location": "pom.xml (log4j 2.14.1)",
      "annual_cost": 780000,
      "fix_effort_weeks": 3,
      "cve": "CVE-2021-44228",
      "confidence": "HIGH"
    },
    {
      "id": "D2",
      "type": "god_library",
      "severity": "critical",
      "location": "commons/PaymentUtils.java",
      "annual_cost": 420000,
      "fix_effort_weeks": 8,
      "confidence": "HIGH"
    },
    {
      "id": "D3",
      "type": "hardcoded_logic",
      "severity": "high",
      "location": "config/AppConfig.java:L45-L67",
      "annual_cost": 312000,
      "fix_effort_weeks": 2,
      "confidence": "HIGH"
    }
  ],
  "recommendation": "REFACTOR_FIRST",
  "reasoning": "PCI-DSS audit will fail due to hardcoded credentials and unpatched"
}

```

Error Handling 1

```

python
def call_claude_with_retry(self, **kwargs):
    """Handle Anthropic API rate limits with exponential backoff."""
    max_retries = 3
    base_delay = 1

    for attempt in range(max_retries):
        try:
            return self.client.messages.create(**kwargs)
        except RateLimitError as e:
            if attempt == max_retries - 1:
                raise
            delay = base_delay * (2 ** attempt)
            logger.warning(f"Rate limited. Retrying in {delay}s...")
            time.sleep(delay)

```

4. References & Code

- **Citations for any sources, papers, or tools referenced**
- GitHub Repository:
<https://github.com/priyanka-patil-tech/technical-debt-negotiator>
- **Link to your demo**
- <https://priyanka-patil-tech.github.io/technical-debt-negotiator/>